

programmingTest2_zsmf

Disclaimer!

Diese Zusammenfassung zielt ab, auf den 2. Programmieretest vorzubereiten. Man sollte diese Datei nicht zum Erlernen verwenden sondern eher als Nachschlagewerk. Vieles in diesem Dokument wurde mit LLMs erstellt und könnte daher Fehler enthalten.

Inhalt

[8 Processes exec pipes](#)

[9 Debugging Sockets](#)

[10 memory-layout code-quality programm-analysis](#)

8_Processes_exec_pipes

Part I: Creating Processes

Einführung in Prozesse

Ein Prozess ist die Repräsentation eines laufenden Programms durch das Betriebssystem (OS).

- Jeder Prozess hat eine **PID** (Process ID).
- Jeder Prozess (außer PID 1) hat einen **Parent Process**.
- Prozesse können verwandte (**Child**) Prozesse erstellen.

☰ Prozessbaum visualisieren

Mit dem Befehl `$ pstree -p 1$` kann die Hierarchie angezeigt werden (z.B. systemd als Root).

```
$ pstree -p 1
systemd(1) -- ModemManager(3270) -- {ModemManager}(3277)
      |                                     | - {ModemManager}(3279)
      |                                     ' - {ModemManager}(3282)
      | - NetworkManager(3167) -- {NetworkManager}(3227)
      |                                     | - {NetworkManager}(3228)
      .
      .
      | - containerd(3465) -- {containerd}(3470)
      |                                     | - {containerd}(3471)
      .
      .
      | - cron(3118)
      | - dbus-daemon(3087)
      | - dockerd(4940) -- {dockerd}(4947)
      |                                     | - {dockerd}(4948)
      .
      .
```

Prozess-Erstellung: `fork(2)`

Der System Call `fork()` erzeugt einen neuen Prozess durch Duplizierung des aufrufenden Prozesses.

Header: `#include <unistd.h>`

Signatur: `pid_t fork(void);`

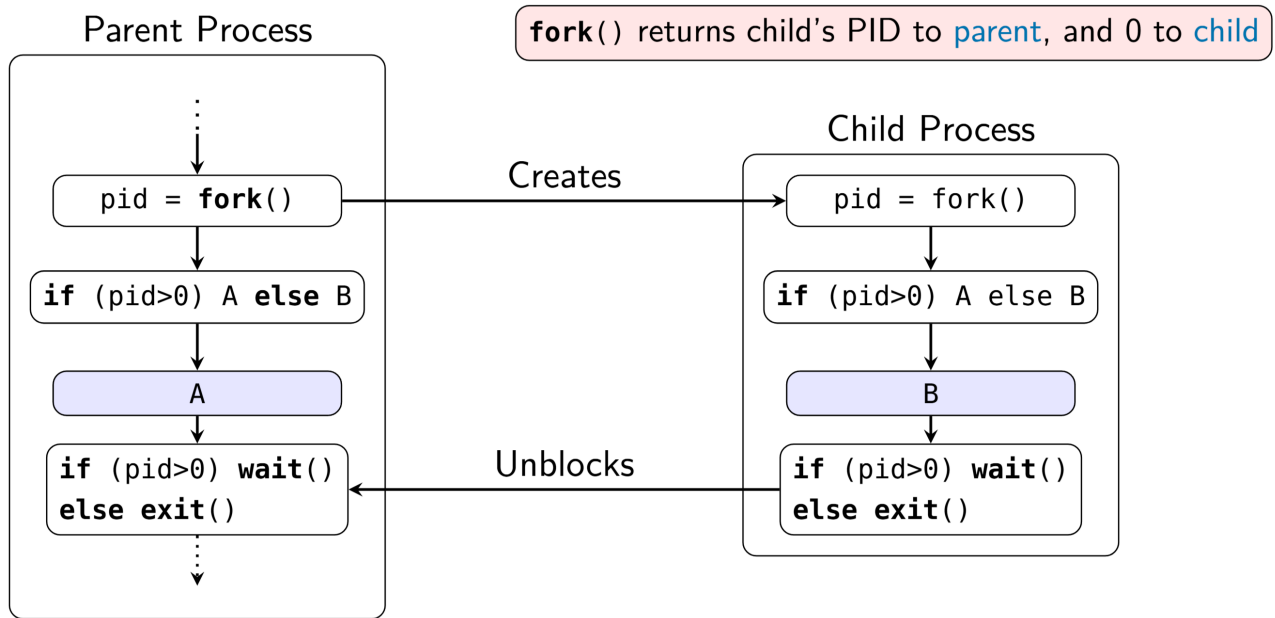
Rückgabewerte:

- `-1`: Fehler (siehe `errno`).
- `0`: Wir befinden uns im **Child Process**.
- `> 0`: Wir befinden uns im **Parent Process** (Rückgabewert ist die PID des Kindes).

📋 Ablauflogik

1. **Parent** ruft `$pid = fork()$` auf.
2. **Parent** erhält `$pid > 0$`.

3. **Child** wird erstellt und erhält `$pid = 0$`.
4. Beide laufen ab der Stelle nach dem `fork()` unabhängig weiter.



Code-Struktur für `fork`

Typisches Pattern

```

1  pid_t pid = fork();
2
3  switch (pid) {
4      case -1:
5          fprintf(stderr, "fork failed: %s\n", strerror(errno));
6          exit(EXIT_FAILURE);
7
8      case 0: /* ... Child instructions ... */ break;
9
10     default: /* ... Parent instructions ... */ break ;
11 }
  
```

Eigenschaften des Child Process

Das Kind "erbt" vieles vom Elternprozess, ist aber ab dem Moment des Forks eigenständig.

Child erbt (Kopie/Gemeinsam)	Unterschied / Unabhängig
Geöffnete Dateien (teilen sich den File Pointer!)	Speicherbereich (Variablen sind unabhängig)
File Buffers	Signal-Handling kann neu konfiguriert werden
Signal Handling (initial)	

Child erbt (Kopie/Gemeinsam)	Unterschied / Unabhängig
Aktuelle Werte von Variablen	

📘 Inter Process Communication (IPC)

Da Variablen in getrennten Speicherbereichen liegen, ist **IPC** notwendig, um Daten zwischen Parent und Child auszutauschen.

Pitfalls (Fallstricke)

Ungeflushter Buffer

Wenn `printf` ohne Newline (`\n`) oder `fflush` vor einem `fork()` ausgeführt wird, befindet sich der Text noch im Buffer. Da der Buffer beim Fork kopiert wird, geben **beide** Prozesse den Inhalt aus.

💣 "HelloHello" Problem

```

1  #include <stdio.h>
2
3  int main(int argc, char **argv) {
4      printf("Hello");
5
6      fork();
7      return 0;
8  }
```

Output: "HelloHello". **Why?**

Answer

Unflushed buffer copied on fork!

Lösung: `fflush(stdout);` vor dem `fork()` aufrufen, um Streams zu leeren.

Prozess-Terminierung: `exit(3)`

Beendet einen Prozess und gibt einen Status zurück.

Signatur: `void exit(int status);`

- **Status:** Konventionell 0-255.
 - `EXIT_SUCCESS`: Erfolgreich.
 - `EXIT_FAILURE`: Fehler aufgetreten.
- **Aktionen bei `exit()`:**
 - Leeren (Flush) und Schließen aller stdio Stream Buffer.
 - Schließen aller offenen File Descriptors.
 - Löschen temporärer Dateien (`tmpfile`).

- Aufruf von Exit-Handlern (`atexit`).

Warten auf Child Processes: `wait(2)`, `waitpid(2)`

Elternprozesse sollten auf die Beendigung ihrer Kinder warten, um Ressourcen freizugeben und den Status abzufragen.

Header: `#include <sys/wait.h>`

- `pid_t wait(int *status);`
 - Blockiert, bis **irgendein** Kind terminiert.
- `pid_t waitpid(pid_t pid, int *status, int options);`
 - Blockiert, bis ein **spezifisches** Kind terminiert (`-1` als PID wartet auf irgendeines).

Status-Macros (zur Auswertung von `*status`):

- `WIFEXITED(status)`: True, wenn Kind normal beendet wurde.
- `WEXITSTATUS(status)`: Gibt den Exit-Code des Kindes zurück.
- `WIFSIGNALED(status)`: Infos über Signale.

Wait Beispiel

```

1  #include <sys/wait.h>
2
3  int status;
4  pid_t child_pid, pid;
5
6  ...
7  pid = wait(&status);
8  if (WEXITSTATUS(status) == EXIT_SUCCESS) { /* child 'pid' succeeded */ }
9
10 pid = waitpid(-1, &status, 0);           // Same as pid = wait(&status)
11
12 ...
13 child_pid = fork();
14 if (child_pid) waitpid(child_pid, &status, 0); // wait for specific child

```

Zombies und Orphans

Zombie Process

Ein Kindprozess, der terminiert ist, aber dessen Elternprozess noch nicht `wait()` aufgerufen hat.

- Der Eintrag bleibt in der Prozesstabelle (belegt Platz/PID).
- Wenn die Tabelle voll ist, können keine neuen Prozesse starten.

Orphan Process

Ein Kindprozess, dessen Elternprozess vor ihm terminiert ist.

- Das Kind wird vom `init` Prozess adoptiert.
- Wenn die Waise terminiert, räumt `init` sie aus der Tabelle auf.

Benachrichtigung bei Terminierung

Wenn der Parent nicht blockieren (warten) will, gibt es zwei Wege:

1. **Synchron (Polling):** `waitpid(-1, &status, WNOHANG)`
 - Kehrt sofort zurück.
 - Gibt `$PID > 0` zurück, wenn ein Kind terminiert ist, sonst `0`.
2. **Asynchron (Signal):**
 - `SIGCHLD` Signal mit einem Handler (`sigaction`) abfangen.
 - `wait` innerhalb des Handlers aufrufen (blockiert garantiert nicht).

Part II: The exec Family

Überblick

Die `exec`-Familie lädt ein **neues Programm** in den **selben Prozess**.

- Die **PID bleibt gleich**, aber der Speicher (Memory Image) wird ersetzt.
- **Kein Return:** Wenn der Aufruf erfolgreich ist, kehrt die Funktion nicht zurück (da der Code überschrieben wurde).
- **Trick:** Kombination mit `fork()` -> Parent forked, Child führt `exec` aus.

Varianten (Suffixe)

Es gibt verschiedene Wrapper für den Systemcall `execve`:

- **l (list):** Argumente als variable Liste übergeben.
- **v (vector):** Argumente als Array (`argv[]`) übergeben.
- **p (path):** Sucht das Programm in der `$PATH` Umgebungsvariable (statt absoluter Pfad).
- **e (environment):** Erlaubt Angabe eines eigenen Environment-Arrays (andere erben vom Caller).

≡ Argumente

- Das erste Argument (`argv[0]`) ist konventionell der Programmname.
- Das letzte Argument **muss** ein `NULL`-Pointer sein.

Exec Beispiele

```
#include <unistd.h>
```

```
char *cmd[] = { "ls", "-l", (char *) 0 };
execv("/bin/ls", cmd);
```

```
execl("/bin/ls", "ls", "-l", NULL);
```

```
int fd = open("/bin/ls", O_RDONLY);
fexecve(fd, cmd, NULL);
```

Note on Arguments

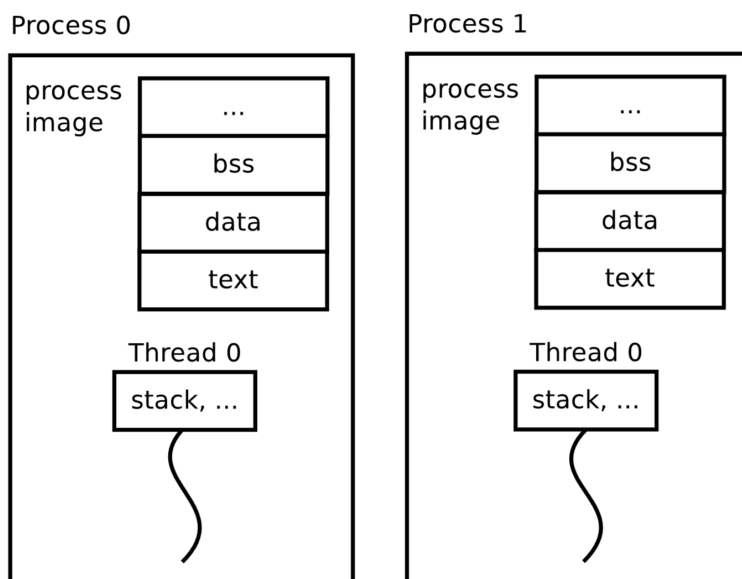
- Provide the full argument list, including argv[0] (program name)
- Last argument must be a NULL pointer!

Exkurs: Prozesse vs. Threads

Feature	Prozess (<code>fork()</code>)	Thread (<code>pthread()</code>)
Speicher	Separater Speicher (Memory Space)	Teilen sich denselben Speicher
Erstellung	Relativ "teuer" für das OS	Relativ "günstig"
Daten	IPC notwendig (Pipes, Shared Mem)	Gemeinsamer Zugriff auf Variablen
Synchronisation	IPC / Semaphoren in Shared Mem	Mutex etc. nötig

🔗 Shared Memory bei Prozessen

Eltern können `mmap(2)` mit `MAP_SHARED` | `MAP_ANONYMOUS` nutzen, um Speicherbereiche zu teilen (z.B. für Semaphoren).

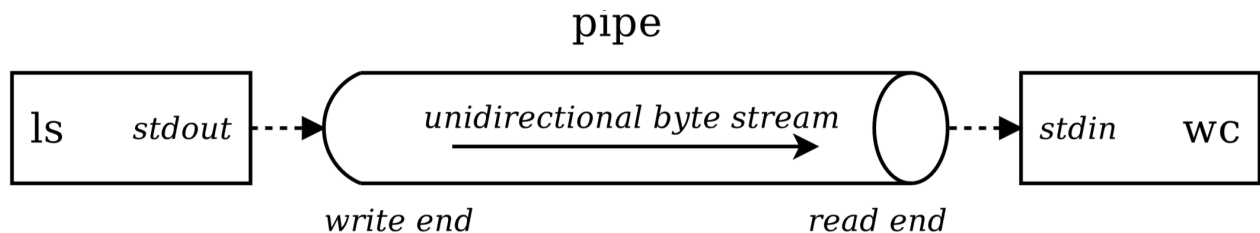


Part III: Pipes

UNIX Pipes Konzept

Eine Pipe ist ein unidirektionaler Byte-Stream zur Kommunikation zwischen Prozessen.

- Beispiel Shell: `$ ls | wc -l$`
- Hat ein **Write End** und ein **Read End**.
- Gepuffert (Buffered).
- **Implizite Synchronisation** (Blockiert bei leer/voll).



Erstellen: `pipe(2)`

Header: `#include <unistd.h>`

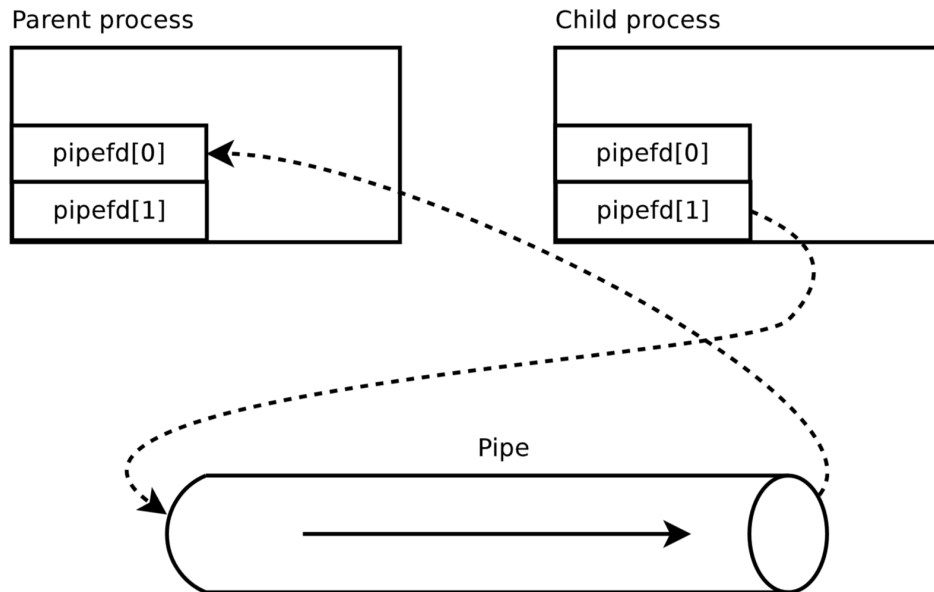
Signatur: `int pipe(int pipefd[2]);`

- Erstellt zwei File Descriptors im Array `pipefd`:
 - `pipefd[0]`: **Read end**
 - `pipefd[1]`: **Write end**

Workflow: Pipe & Fork

Um zwischen Parent und Child zu kommunizieren:

1. **Pipe erstellen** (vor dem Fork!).
2. `fork()` **aufrufen**.
3. **Nicht benötigte Enden schließen**:
 - Wenn Parent liest und Child schreibt:
 - Parent schließt `pipefd[1]` (Write).
 - Child schließt `pipefd[0]` (Read).
4. Kommunizieren (`read`/`write` oder `fdopen`).



Implizite Synchronisation & Regeln

- `read()` blockiert, wenn die Pipe leer ist.
- `read()` gibt `0` (EOF) zurück, wenn **alle** Write-Ends geschlossen sind.
- `write()` blockiert, wenn der Pipe-Buffer voll ist.
- `write()` erzeugt Signal `SIGPIPE` (und Error `EPIPE`), wenn **alle** Read-Ends geschlossen sind.

🔥 Wichtig

Nicht genutzte Pipe-Enden **müssen** geschlossen werden, damit EOF korrekt erkannt wird und Prozesse nicht ewig warten (Deadlock).

Redirection (Umleitung von stdin/stdout)

Szenario: Parent möchte Output vom Child abfangen, Child nutzt aber `printf` (schreibt nach stdout).

Lösung: `dup2`

- `int dup2(int oldfd, int newfd);`
- Dupliziert `oldfd` auf `newfd`. Wenn `newfd` offen war, wird er vorher geschlossen.

Ablauf im Child:

1. Schließe Read-End der Pipe.
2. `dup2(pipefd[1], STDOUT_FILENO);` -> stdout zeigt jetzt in die Pipe.
3. Schließe das ursprüngliche `pipefd[1]` (da jetzt dupliziert).
4. `exec(...)` -> Das neue Programm schreibt nun unwissentlich in die Pipe.

🔗 Redirection Code Beispiel

```

1  int pipefd[2]; char buffer[1024]; FILE *stream;
2  pipe(pipefd);
3
4  switch(fork()) {
5      case -1: /* error handling */ break;
6      case 0:      close(pipefd[0]);           // Close read end
7                  dup2(pipefd[1], STDOUT_FILENO);
8                  close(pipefd[1]);           // Close old fd
9                  execlp("ls", "ls", "-l", NULL); break;
10     default:      close(pipefd[1]);           // Close write end
11                  stream = fdopen(pipefd[0], "r");
12                  while (fgets(buffer, sizeof buffer, stream)) { /* ... */ }
13                  fclose(stream); wait();
14  }

```

Pitfalls & Tipps für die Übung

⚠ Deadlocks

Bei 2-Way-Communication (Prozess A <-> Prozess B) braucht man **zwei Pipes**.
 Wenn beide versuchen zu lesen und beide Pipes leer sind -> Deadlock.

🔗 Performance / Design

- **Nicht:** Parent liest ganze Datei in ein Array -> Fork -> Child verarbeitet Kopie. (Ineffizienter Speicherverbrauch).
- **Sondern (Pipeline):**
 1. Parent erstellt Pipe & Fork.
 2. Parent liest Datei Zeile für Zeile (`fgets`).
 3. Parent schickt Zeile in die Pipe.
 4. Child verarbeitet Zeile sofort, wenn sie ankommt.
 -> Lesen und Verarbeiten passieren parallel!

📁 Named Pipes (FIFO)

- Erstellt mit `mkfifo`.
- Existieren als Datei im Dateisystem.
- Lesen von leerer Pipe blockiert (anders als bei normaler Datei, wo EOF käme).

9_Debugging_Sockets

Part I: Introduction to GDB

Der **GDB** (GNU Project Debugger) ermöglicht die Inspektion eines Programms während der Laufzeit.

Voraussetzungen

Damit Debugging funktioniert, muss das Programm mit **Debugging Symbols** kompiliert werden.

- Mapping von Ausführungspunkten zu Quellcodezeilen.
- Mapping von Speicheradressen zu Variablennamen.
- **Flag:** `gcc -g` (Deaktiviert typischerweise Optimierungen).

GDB Basics

- **Starten:** `$ gdb ./myprogram`
- **TUI Mode (Rich Text Interface):** `$ gdb -tui ./myprogram` oder Argument `-tui` hinzufügen.

Wichtige Befehle im GDB Prompt `(gdb)`:

Befehl	Beschreibung
<code>run [args]</code>	Startet einen neuen Prozess (mit Argumenten).
<code>kill</code>	Beendet den mit <code>run</code> gestarteten Prozess.
<code>attach <pid></code>	Hängt sich an einen existierenden Prozess an.
<code>detach</code>	Löst sich vom Prozess.
<code>help</code>	Inline Hilfe.
<code>quit</code>	Beenden.

Breakpoints

Zeilen, an denen GDB die Ausführung pausieren soll.

- **Setzen:**
 - Datei und Zeile: `break example.c:24`
 - Funktionsanfang: `break main`
 - **Bedingt:** `break <...> if i==10` (Pausiert nur, wenn Bedingung wahr ist).
- **Verwalten:**
 - Auflisten: `info breakpoints`
 - Manipulieren: `disable`, `enable`, `delete <number>`

Execution Control

Wenn GDB pausiert ist (z.B. am Breakpoint):

- **Status:** `where` oder `bt` (Backtrace/Callstack anzeigen).
- **Stepping:**
 - `next`: Eine Zeile weiter (überspringt Funktionsaufrufe/führt sie komplett aus).
 - `step`: Eine Zeile weiter (springt **in** Funktionen hinein).
 - `stepi`: Einen einzelnen Maschinenbefehl weiter.
- **Fortsetzen:**
 - `continue`: Weiterlaufen bis zum nächsten Breakpoint.
 - `until <file:line>`: Weiterlaufen bis bestimmte Stelle erreicht.
 - `finish`: Weiterlaufen bis aktuelle Funktion zurückkehrt (return).

Variablen & Watchpoints

Arbeiten mit Variablen

- **Wert drucken:** `print <expr>`
- **Wert ändern:** `set variable $x=0$`
- **Dauerhaft anzeigen:** `display <expr>` (Druckt Ausdruck bei jedem Break).

Watchpoints:

Pausieren die Ausführung, wenn auf eine Variable zugegriffen wird (kann langsam sein ohne HW-Support).

- `watch myvar`: Bei Schreibzugriff (Write).
- `rwatch myvar`: Bei Lesezugriff (Read).
- `awatch myvar`: Bei beidem (Access).

Signals & Forks

- **Signale:**
 - `Ctrl-C` sendet `SIGINT` → gibt Kontrolle an Debugger zurück (terminiert Programm nicht!).
 - Handling anpassen: `handle <signal> [stop/nostop] [print/noprint] ...`
 - Manuelles Senden: `signal <signal>`
- **Debugging mit `fork()`:**
 - Standardmäßig folgt GDB dem Parent.
 - Einstellung ändern: `set follow-fork-mode [child|parent]` (muss **vor** dem Fork-Aufruf gesetzt werden).

Conditional Compilation (Debug Messages)

Nützlich, um Debug-Code im Release zu deaktivieren.

```
#ifdef DEBUG
    // ... setup debug macro ...
#else
    #define debug(msg) /* NOP */
#endif
```

- Kompilieren mit `-DDEBUG`, um Nachrichten zu aktivieren.
-

Part II: Sockets

IPC Methoden (Inter-Process Communication)

- **Explizite Synchronisation:** Shared Memory, Semaphoren (meist un-related processes).
- **Implizite Synchronisation:** Pipes (related processes).
- **Sockets:**
 - Implizite Synchronisation zwischen **unrelated processes**.
 - Möglich auf verschiedenen Maschinen (Netzwerk).
 - File-like Interface für Transport-Layer Services.

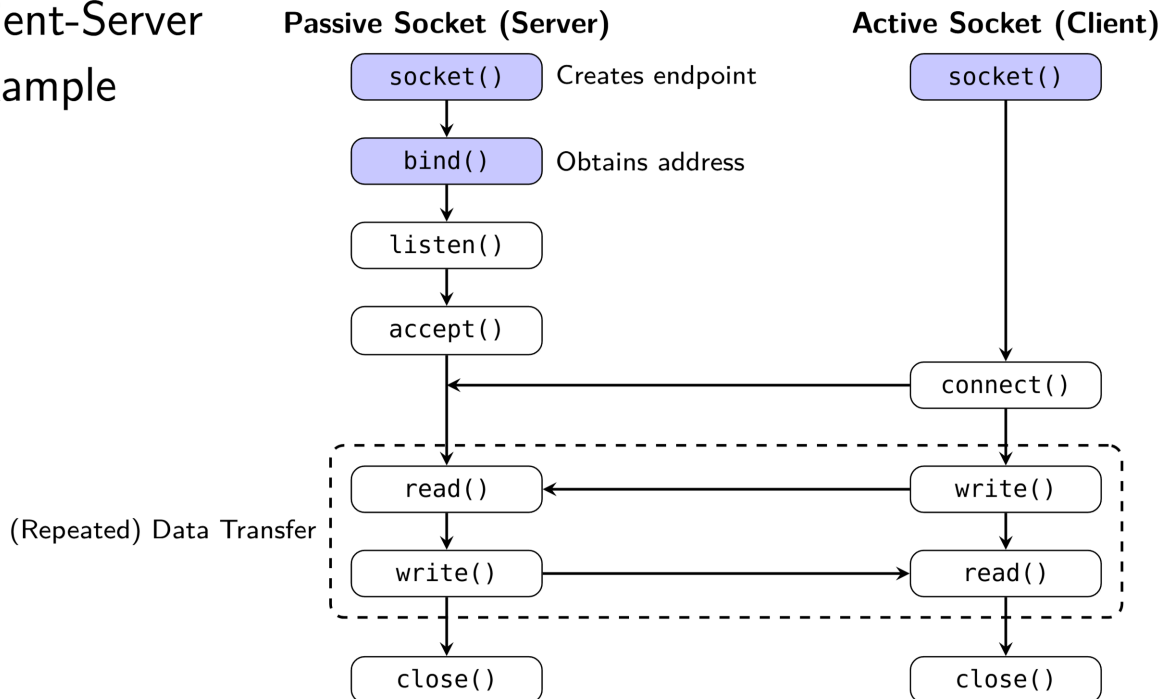
Transport Layer Protokolle

Protokoll	Eigenschaft	Beschreibung
TCP	Stream-based	Verbindungsaufbau nötig, zuverlässiger Datentransfer.
UDP	Message-based	Verbindungslos, fixe Nachrichtengröße, Verlust möglich.
UNIX	Local IPC	Lokal auf dem System, stream- oder message-based.

Client-Server Lifecycle (TCP)

Hier ist der Ablauf der Systemaufrufe für Server (Passive) und Client (Active).

Client-Server Example



Socket Setup Funktionen

1. `socket(2)` - Endpoint erstellen

Erstellt einen Endpoint für die Kommunikation.

```
int socket(int family, int type, int protocol);
```

- **family:**
 - `AF_INET` (IPv4)
 - `AF_INET6` (IPv6)
 - `AF_UNIX` (Local)
- **type:**
 - `SOCK_STREAM` (Verbindungsorientiert → TCP)
 - `SOCK_DGRAM` (Verbindungslos → UDP)
- **protocol:** `0` für Default.
- **Return:** File Descriptor (oder -1 bei Fehler).

2. `bind(2)` - Adresse zuweisen

Weist dem Socket eine Adresse (IP + Port) zu. (Meistens Serverseitig).

```
int bind(int socket, struct sockaddr *address, socklen_t addr_len);
```

✗ Die `sockaddr` Problematik

`struct sockaddr` ist generisch. Die tatsächliche Struktur hängt von der Family ab (z.B. `struct sockaddr_in` für IPv4).

- Casting notwendig.
- **Byte Order:** Unterscheidung zwischen Machine Endianness und **Network Byte Order** (immer Big Endian).

- Konvertierung nötig: `htonl`, `ntohl` etc.

→ **Lösung:** Helper-Funktionen nutzen (`getaddrinfo`).

3. `getaddrinfo(3)` - Die moderne Lösung

Bereitet Socket-Strukturen vor, abstrahiert IP-Versionen und Byte-Order.

```
int getaddrinfo(const char *node, const char *service, const struct addrinfo
*hints, struct addrinfo **res);
```

- `node`: Hostname oder IP.
- `service`: Port ("80") oder Service Name ("http").
- `hints`: Kriterien zur Filterung (z.B. nur TCP).
- `res`: Output (Linked List von Ergebnissen).
- **Return:** 0 bei Erfolg (Fehlercode via `gai_strerror` lesbar machen).
- **Cleanup:** `freeaddrinfo(res)` nicht vergessen!

⚡ Veraltet: `gethostbyname`

Verwende niemals `gethostbyname` (unterstützt z.B. kein IPv6). In der Übung/Prüfung verboten!

Code-Beispiel `getaddrinfo` Setup:

`getaddrinfo`: Example²

```
1  int status, sockfd;
2  struct addrinfo hints;
3  struct addrinfo *servinfo;
4
5  memset(&hints, 0, sizeof hints); // clear hints structure
6  hints.ai_family = AF_UNSPEC;      // don't care, IPv4 or IPv6
7  hints.ai_socktype = SOCK_STREAM; // TCP
8  hints.ai_flags = AI_PASSIVE;      // Let system figure out the address
9
10 if ((status = getaddrinfo(NULL, "3490", &hints, &servinfo)) != 0) {
11     fprintf(stderr, "Error: %s\n", gai_strerror(status)); exit(EXIT_FAILURE);
12 }
13
14 sockfd = socket(servinfo->ai_family, servinfo->ai_socktype, servinfo->ai_protocol);
15 bind(sockfd, servinfo->ai_addr, servinfo->ai_addrlen);
```

Verbindungsaufbau

`listen(2)` (Server)

Markiert den Socket als passiv (wartet auf Verbindungen).

```
int listen(int socket, int backlog);
```

- `backlog`: Größe der Warteschlange für pending connections.
- Nur für Connection-oriented Sockets (TCP).

`connect(2)` (Client)

Initiiert Verbindung zum Server.

```
int connect(int socket, const struct sockaddr *address, socklen_t addr_len);
```

- Blockiert, bis Verbindung steht oder Fehler auftritt.

`accept(2)` (Server)

Nimmt eine Verbindung an.

```
int accept(int socket, struct sockaddr address, socklen_t addr_len);
```

- Blockiert bis Request kommt.
- **Wichtig:** Gibt einen **neuen** File Descriptor zurück, der für die eigentliche Kommunikation genutzt wird. Der ursprüngliche Socket hört weiter zu (Listening Socket).

Datenaustausch

Sobald Verbindung steht: Nutzung von `read()` und `write()`.

- `read`: Blockiert bei leerem Socket.
- `write`: Blockiert bei vollem Buffer.

≡ Stream I/O mit FILE*

Man kann den Socket-FD in einen FILE* Stream wrappen, um Funktionen wie `fprintf` oder `fgets` zu nutzen.

```
FILE *sockfile = fdopen(sockfd, "r+");
```

Achtung: Flushing beachten (`fflush`)!

Code-Beispiel Stream I/O:

Stream I/O on Sockets - Example

```

1  /* ... (set up hints) */
2  int res = getaddrinfo("noss1.sh", "http", &hints, &ai);
3  int sockfd = socket(ai->ai_family, ai->ai_socktype, ai->ai_protocol);
4  if (sockfd < 0) { /* ... */}
5  if (connect(sockfd, ai->ai_addr, ai->ai_addrlen) < 0) { /* ... */ }
6
7  FILE *sockfile = fdopen(sockfd, "r+");
8
9  if (fputs("GET / HTTP/1.1\r\nHost: noss1.sh\r\n\r\n", sockfile) == EOF) { /* ... */ }
10 if (fflush(sockfile) == EOF) { /* ... */}
11
12 char buf[1024];
13
14 // print the reply:
15 while (fgets(buf, sizeof(buf), sockfile) != NULL) fputs(buf, stdout);

```

Spezielle Send/Recv Funktionen

Alternativen zu read/write mit zusätzlichen Flags.

- `send(..., int flags)`
- `recv(..., int flags)`
- **Flags:**
 - `MSG_DONTWAIT`: Non-blocking (Return -1 & `errno=EAGAIN`).
 - `MSG_WAITALL`: Wartet bis Buffer voll ist (bei `recv`).

Socket Options: `setsockopt(2)`

Einstellungen am Socket vornehmen.

Wichtiges Beispiel: `SO_REUSEADDR`

Verhindert "Address already in use" Fehler beim Neustart des Servers (wenn Port noch im `TIME_WAIT` Status ist).

```

int optval = 1;
setsockopt(serverfd, SOL_SOCKET, SO_REUSEADDR, &optval, sizeof(optval));

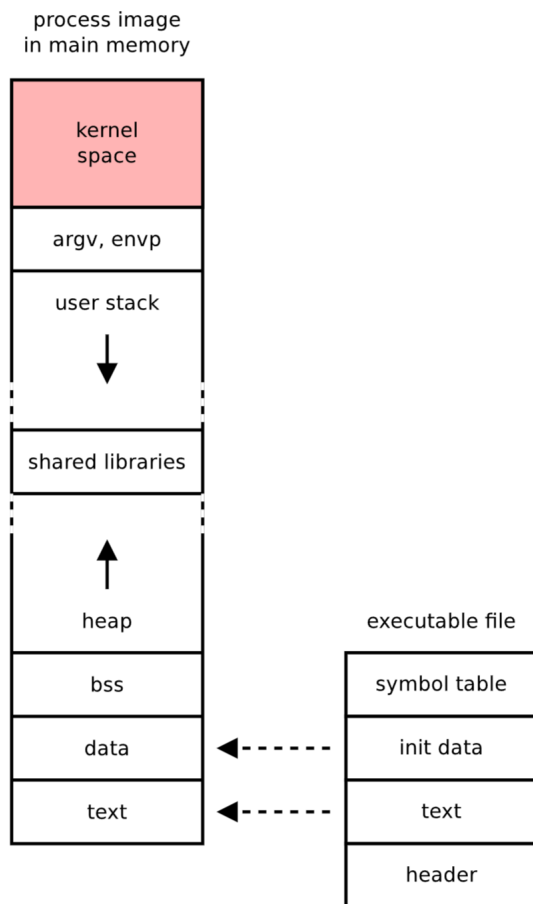
```

10_memory-layout_code-quality_programm-analysis

Part V: Memory Layout

Memory Layout eines Prozesses

Das Speicherlayout eines laufenden Prozesses (Process Image) im Hauptspeicher ist in verschiedene Segmente unterteilt, die unterschiedliche Eigenschaften haben.



ⓘ Speicherregionen (grob)

- **Stack:** Lokale Variablen
- **Heap:** Dynamisch allozierter Speicher
- **BSS:** Globale/statische Variablen, die **implizit** mit 0 initialisiert sind ("*Block Starting Symbol*").
- **Data:** Globale/statische Variablen, die **explizit** initialisiert wurden.
- **Text:** String-Konstanten und Instruktionen (Programmcode). **Read-only**.

- **Dynamische Größe:** Stack und Heap können ihre Größe zur Laufzeit verändern (wachsen aufeinander zu).
- **Kernel Space:** Oberhalb des User-Stacks (für Nutzer nicht direkt zugreifbar).

- **Executable File:** Enthält Symbol Table, Init Data, Text und Header, die in den Speicher gemappt werden.

Globale Variablen: Data vs. Text Segment

Es gibt einen wichtigen Unterschied bei der Deklaration von String-Literalen:

```
1 char hello[] = "hello";
2
3 void f(void) {
4     hello[0] = 'H'; // Will it work?
5 }
```

OK!

hello is an initialized array in the **data** segment.

```
1 char *hello = "hello";
2
3 void f(void) {
4     hello[0] = 'H'; /* Will it work? */
5 }
```

Memory access error

hello is a pointer (in the data segment) to a string constant "hello" in the **text** segment (which is read-only).

1. **Array:** `char hello[] = "hello";`

- `hello` ist ein initialisiertes Array im **Data Segment**.
- Schreibzugriff (`hello[0] = 'H'`) ist **OK**.

2. **Pointer:** `char *hello = "hello";`

- `hello` ist ein Pointer (im **Data Segment**), der auf eine String-Konstante im **Text Segment** zeigt.
- Das Text Segment ist **Read-only**.
- Schreibzugriff (`hello[0] = 'H'`) führt zu **Memory Access Error** (Segfault).

Program Stack

- **Funktionsweise:**
 - Initial leer.
 - Beim Betreten eines Blocks (z.B. Funktion `main`): Alle lokalen Variablen des Blocks werden auf den Stack gelegt.
 - **Initialisierung:** Wert durch Ausdruck bestimmt; ohne Initialisierung ist der Wert **zufällig/arbiträr**.
 - Beim Verlassen des Blocks: Variablen werden vom Stack entfernt.

Risiken mit lokalen Variablen (Dangling Pointers)

Es ist ein schwerer Fehler, Pointer auf lokale Variablen zurückzugeben, da diese nach Funktionsende nicht mehr existieren.

```

1  int *g(int x)
2  {
3      int y = x + 10;
4      return &y;
5  }
6
7  int main(void)
8  {
9      int *a; // <--
10     a = g(1);
11     ...
12 }

```

⚠ Undefined Behavior

Wenn eine Funktion `g(x)` die Adresse einer lokalen Variable `y` zurückgibt (`return &y;`), ist der Zugriff auf diesen Pointer in `main` **Undefined Behavior**. Der Speicherbereich auf dem Stack (z.B. Adresse `0xff4`) ist nach dem `return` nicht mehr für `y` reserviert und kann überschrieben werden.

Stack Overflow (SO)

- Der Stack hat eine begrenzte Größe (Linux Default oft 8MB, prüfbar mit `ulimit -s`).
- **Ursache:** Wenn das Limit überschritten wird, terminiert das OS das Programm.
- Häufig bei tiefer oder unendlicher **Rekursion**.

Beispiel Rekursion (Fakultät):

$$n! = n \cdot (n - 1)! \quad \text{für } n \geq 1, \quad 0! = 1$$

```

1  // NB This will return 0 for n >= 66,
2  // but this is beside the point.
3  uint64_t fac(uint64_t n) {
4      if (n == 0) { return 1; }
5      return n * fac(n - 1);
6  }

```

- Jeder Aufruf `fac(n)` benötigt eigenen Stack-Space für:
 - Return Address (nächste Instruktion).
 - Argumente und lokale Variablen.
- Speicher wird erst beim `return` freigegeben.
- Bei $n \approx 20.000$ erfolgt oft ein **SIGSEGV** (Stack Overflow).

Tail Call Optimization (Endrekursion)

Ein rekursiver Aufruf ist die **allerletzte** Aktion in einer Funktion.

Vorteil:

- Es muss kein Stack-Frame der aktuellen Funktion bewahrt werden.
- Der Compiler kann dies erkennen (**Tail Call Optimization**).
- Der Aufruf wird intern in einen **Sprung (Jump)** zum Anfang der Funktion umgewandelt (effektiv eine Schleife).
- **Kein zusätzlicher Stack-Verbrauch = Kein Stack Overflow.**

```
1 uint64_t fac_tail(uint64_t n, uint64_t acc) {  
2     if (n == 0) { return acc; }  
3     return fac_tail(n - 1, n * acc);  
4 }
```

Dynamic Memory (Heap)

- Verwendung, wenn Speicherbedarf zur Compile-Zeit unbekannt ist.
- **Funktionen:**
 - `malloc`, `calloc`: Reservieren Speicher.
 - `realloc`: Verändert Größe eines Blocks.
 - `free`: Gibt Speicher frei.
 - (FYI) `alloca`: *Alloziert dynamisch auf dem **Stack** (C99 VLA).*

⚡ Pitfalls & UB

- Double Free (doppelte Freigabe).
- Use-After-Free (Zugriff nach Freigabe).
- `free(x)` darf nur auf Zeiger angewendet werden, die von malloc/calloc/realloc stammen.

Security Aspekte (CWE-226)

Wenn sensibler Speicher (z.B. Passwörter, Keys) freigegeben wird (`free`), werden die Daten **nicht automatisch gelöscht**.

```

1  char *secret_key;
2
3  secret_key = malloc(sizeof(char) * 128);
4  load_key(secret_key);
5  /* use key for encryption, etc. */
6
7  // erase key (or: memset(secret_key, 0, 128); )
8  for (int i = 0; i < 128; i++) secret_key[i] = '\0';
9  free(secret_key);

```

- **Problem:** Ein einfacher Loop oder `memset` zum Nullen des Speichers *vor* dem `free` könnte vom Compiler weg-optimiert werden ("Dead Store Elimination"), da der Speicher danach nicht mehr benutzt wird (laut C-Standard).
- **Lösung:** Spezielle Funktionen wie `OPENSSL_cleanse(3)` nutzen, die garantiert nicht weg-optimiert werden.
- Gilt auch für `realloc`: Kopiert evtl. Daten, ohne das Original zu löschen.

Part VI: Code Quality & Program Analysis

Coding Guidelines vs. Style Guides

Typ	Fokus	Beispiele
Coding Guidelines	Syntax-Subset, Fehlervermeidung, Analysierbarkeit	Keine Rekursion, kein <code>goto</code> , defensive Programmierung
Style Guides	Formatierung, Benennung, Lesbarkeit	Einrückung, Variablennamen (camelCase vs snake_case)

Beispiel: NASA JPL Coding Standard

- Maximal eine Bildschirmseite pro Funktion.
- Eingeschränkte Nutzung des Präprozessors.
- **Defensive Programming:** Assertion-Nutzung, Rückgabewerte prüfen.
- Kein `goto`, keine Rekursion (wegen Stack-Analyse).
- Keine dynamische Speicherallokation *nach* der Initialisierungsphase.

Assertions (`assert(3)`)

- `#include <assert.h>`
- `assert(expr)`: Wenn `expr` 0 (falsch) ist, bricht das Programm ab (`abort(3)`), ggf. mit Core Dump.
- Kann mit Compiler-Flag `-DNDEBUG` deaktiviert werden (für Release-Builds).

```

1  #include<assert.h>
2  void add_address_to_list(const char *street, unsigned int no) {
3      assert(strlen(street) < 40);          /* street must have < 40 characters */
4      item_t *item = malloc(sizeof *item);
5
6      /* item pointer must be valid (&& "..." just for info if it fails) */
7      assert(item != NULL && "item pointer is NULL");
8
9      assert(list_contains(street, no));    /* Given address must belong to a list */
10 }

```

Secure Coding (ISO/IEC TS 17961)

- **Tainted Data:** Daten vom User (argv, stdin, Netzwerk) müssen validiert/sanitized werden.
- Vermeidung von Undefined Behavior (Out-of-Bounds, Use-After-Free).

Program Analysis

Wie prüft man die Einhaltung von Regeln jenseits von Syntax?

1. Static Analysis

- Analyse des Codes **ohne Ausführung**.
- Baut ein Modell des Programms.
- **Tools:** `cppcheck`, `Frama-C`, `gcc -fanalyzer`.
- **Trade-off:** Oft unvollständig oder nicht "sound" (wegen Entscheidbarkeitsproblemen).

2. Dynamic Analysis

- Analyse **während der Ausführung**.
- Code wird "instrumentiert" (zusätzlicher Code eingefügt).
- **Tools:** `AddressSanitizer` (ASan), `valgrind`.
- Nachteil: Prüft nur den ausgeführten Pfad (Testabdeckung wichtig).

Soundness vs. Completeness

Definitionen

- **False Positive (FP):** Bug gemeldet, Code ist aber korrekt.
- **False Negative (FN):** Kein Bug gemeldet, Code ist aber fehlerhaft.
- **Sound (Korrektheit):** Keine False Negatives (findet **alle** Bugs, meldet aber evtl. falsche Alarmer).
- **Complete (Vollständigkeit):** Keine False Positives (jeder gemeldete Bug ist echt).

- Statische Analyse opfert meist Completeness für Soundness (oder ist beides nicht).

- Dynamische Analyse ist meist Complete (meldet nur echte Fehler), aber nicht Sound (findet nicht alles).

Bug Examples & Detection Tools

Wir betrachten ein laufendes Beispiel (`malloc` eines Strings und Manipulation).

Bug 1: Ignorierter `malloc` Fehler

Der Rückgabewert von `malloc` kann `NULL` sein (Out of Memory). Wird dies nicht geprüft, droht Dereferenzierung von `NULL`.

```

1  #include <stdio.h>
2  #include <stdlib.h>
3
4  int main(void) {
5      char *test = malloc(20);
6      //if (test == NULL) { return 1; } <-----
7      for (int i = 0; i < 20; ++i) {
8          test[i] = 'a' + i;
9      }
10     test[19] = '\0';
11     printf("test[10] = %c\n", test[10]);
12     printf("test = %s\n", test);
13     free(test);
14     return 0;
15 }
```

- Compiles with no warnings, even with `-Wall`
- Compiling with `-fanalyzer`: (next slide)

Erkennung:

- `gcc -Wall`: Keine Warnung.
- `gcc -fanalyzer`: Meldet "dereference of possibly-NULL 'test' [CWE-690]".

Bug 2: Out-of-Bounds Access

Schleife läuft zu weit: `for (int i=0; i<=20; ++i)` bei Größe 20 (Indizes 0..19).

```

1  #include <stdio.h>
2  #include <stdlib.h>
3
4  int main(void) {
5      char *test = malloc(20);
6      if (test == NULL) { return 1; }
7      for (int i = 0; i <= 20; ++i) { // <-----
8          test[i] = 'a' + i;
9      }
10     test[19] = '\0';
11     printf("test[10] = %c\n", test[10]);
12     printf("test = %s\n", test);
13     free(test);
14     return 0;
15 }
```

- `gcc -fanalyzer` gives no warnings
- Using `cppcheck`:

```

$ cppcheck main2.c
Checking main2.c ...
main2.c:8:9: error: Array 'test[20]'
accessed at index 20, which is out of
bounds. [arrayIndexOutOfBounds]   test
[i] = 'a' + i;
^
main2.c:7:21: note: Assuming that condition
'i<=20' is not redundant
for (int i = 0; i <= 20; ++i) { // <-----
^
main2.c:8:9: note: Array index out of bounds
test[i] = 'a' + i;
```

Erkennung:

- `gcc -fanalyzer`: Keine Warnung.
- `cppcheck`: Meldet "Array 'test[20]' accessed at index 20".
- **AddressSanitizer (ASan)**: Meldet zur Laufzeit "heap-buffer-overflow".

Bug 3: Use-After-Free (Einfach)

Zugriff auf `test` nachdem `free(test)` aufgerufen wurde.

```

1  #include <stdio.h>
2  #include <stdlib.h>
3
4  int main(void) {
5      char *test = malloc(20);
6      if (test == NULL) { return 1; }
7      for (int i = 0; i < 20; ++i) {
8          test[i] = 'a' + i;
9      }
10     test[19] = '\0';
11     printf("test[10] = %c\n", test[10]);
12     free(test);           // <-----
13     printf("test = %s\n", test); // <-----
14     return 0;
15 }
```

Compiling with `gcc -Wall`:

```

main2.c: In function 'main'
main2.c:12:3: warning: pointer 'test' used
      after 'free' [-Wuse-after-free]
    12 |     printf("test = %s\n", test);
        |     ^~~~~~
main1.c:11:3: note: call to 'free' here
    11 |     free(test);
        |     ^~~~~~
```

- GCC does some very lightweight static analysis with `-Wall`
- (Surprisingly, `-fanalyzer` says nothing)

Erkennung:

- `gcc -Wall`: Erkennt dies überraschenderweise (statische Analyse im Compiler).
- `gcc -fanalyzer`: Meldet hier nichts (laut Folien).

Bug 4: Use-After-Free (Komplex/Konditional)

Der Zugriff erfolgt nur unter einer Bedingung oder ist verschachtelt, sodass einfache statische Analyse versagt.

```

1  #include <stdio.h>
2  #include <stdlib.h>
3
4  int main(void) {
5      char *test = malloc(20);
6      if (test == NULL) { return 1; }
7      for (int i = 0; i < 20; ++i) {
8          test[i] = 'a' + i;
9      }
10     test[19] = '\0';
11     printf("test[10] = %c\n", test[10]);
12     if (test[0] == 'a') { free(test); } // <--
13     printf("test = %s\n", test);
14     return 0;
15 }
```

- Undetected by static analyzers seen so far (`gcc -fanalyzer`, `cppcheck`)
- More advanced tools (e.g., `esbmc`^a) can catch it
- These tools may take extremely long time on large programs

^a<https://esbmc.org/>

- Statische Tools (`gcc`, `cppcheck`) finden dies oft nicht.
- Erfordert fortgeschrittene Model Checker (z.B. `esbmc`) oder dynamische Analyse.

AddressSanitizer (ASan)

- Instrumentiert Speicherzugriffe zur Compile-Zeit.
- Bricht Programm beim ersten Fehler ab.
- Verwendung: `gcc -g -fsanitize=address main.c`
- **Output:** Zeigt genaue Speicheradressen, Stack Traces für Allokation (`malloc`), Freigabe (`free`) und den fehlerhaften Zugriff (WRITE/READ).

```
$ gcc -g -fsanitize=address main2.c
$ ./a.out
```

```
=====
==63052==ERROR: AddressSanitizer: heap-buffer-overflow on address 0x503000000054 at pc 0x6375d31cf2c6 bp 0x7ffd4225bc90 sp 0
x7ffd4225bc80
WRITE of size 1 at 0x503000000054 thread T0
#0 0x6375d31cf2c5 in main /home/luca/osue-backup/asan/main2.c:8
#1 0x7aada142a1c9 in __libc_start_call_main ../sysdeps/nptl/libc_start_call_main.h:58
#2 0x7aada142a28a in __libc_start_main_impl ../csu/libc-start.c:360
#3 0x6375d31cf184 in _start (/home/luca/osue-backup/asan/a.out+0x1184) (BuildId: ac648defb04c0f8f4636dde629f038ad8fed9a84)

0x503000000054 is located 0 bytes after 20-byte region [0x503000000040,0x503000000054)
allocated by thread T0 here:
#0 0x7aada18fd9c7 in malloc ../../src/libsanitizer/asan/asan_malloc_linux.cpp:69
#1 0x6375d31cf25e in main /home/luca/osue-backup/asan/main2.c:5
#2 0x7aada142a1c9 in __libc_start_call_main ../sysdeps/nptl/libc_start_call_main.h:58
#3 0x7aada142a28a in __libc_start_main_impl ../csu/libc-start.c:360
#4 0x6375d31cf184 in _start (/home/luca/osue-backup/asan/a.out+0x1184) (BuildId: ac648defb04c0f8f4636dde629f038ad8fed9a84)
...
```

```
$ gcc -g -fsanitize=address main4.c
$ ./a.out
```

```
test[10] = k
=====
==63400==ERROR: AddressSanitizer: heap-use-after-free on address 0x503000000040 at pc 0x767b5f4a1a6a bp 0x7ffea2078460 sp 0
x7ffea2077bd8
READ of size 2 at 0x503000000040 thread T0
#0 0x767b5f4a1a69 in printf_common ../../src/libsanitizer/sanitizer_common/sanitizer_common_interceptors_format.inc
:563
#1 0x767b5f4ce33b in vprintf ../../src/libsanitizer/sanitizer_common/sanitizer_common_interceptors.inc:1644
#2 0x767b5f4d0180 in printf ../../src/libsanitizer/sanitizer_common/sanitizer_common_interceptors.inc:1702
#3 0x5a0709e363cf in main /home/luca/osue-backup/asan/main4.c:14
...

0x503000000040 is located 0 bytes inside of 20-byte region [0x503000000040,0x503000000054)
freed by thread T0 here:
#0 0x767b5f4fc4d8 in free ../../src/libsanitizer/asan/asan_malloc_linux.cpp:52
#1 0x5a0709e363b4 in main /home/luca/osue-backup/asan/main4.c:12
...

previously allocated by thread T0 here:
#0 0x767b5f4fd9c7 in malloc ../../src/libsanitizer/asan/asan_malloc_linux.cpp:69
#1 0x5a0709e3625e in main /home/luca/osue-backup/asan/main4.c:5
...
```